NLP подходи за категоризация на алгоритмични задачи на български език

Теодор Светославов Костадинов
ф.н. 5MI3400816

Подходи за обработка на естествен език, зимен семестър 2025/2026

1. Избор на задача (Мотивация)

Проблем: Всяко година от 2018 насам в курса по СДА се добавят около 30 нови задачи. Подготовката на студентите и намиране на подобни задачи става все по-трудно.

Подобряване на достъпа до материалите за студенти в СДА курса.

1. Избор на задача

Извличането и класифицирането на задачите са ключови части от решението на нашия проблем.

2. Избор на корпус и допълнителни данни (Данни)

Публичен исторически списък на всички състезания давани по СДА от 2018 насам:

Линк

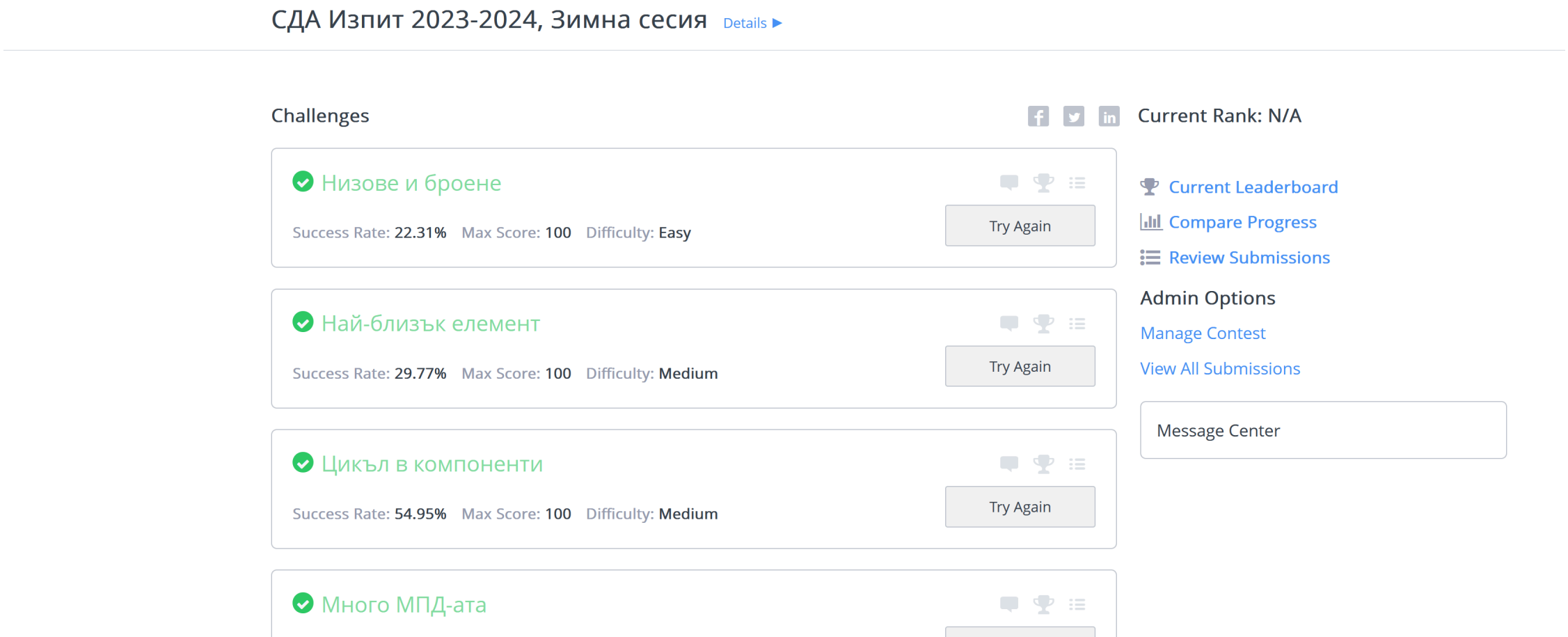
Как изглеждат данните?

Първоначалния документ конвертиран към CSV:

```
You, 3 weeks ago | 1 author (You)
year,type,link          You, 3 weeks ago • [IR] Add all links …
2018/2019,homework,www.hackerrank.com/practice-1-si
2018/2019,homework,www.hackerrank.com/si-practice-2
2018/2019,homework,www.hackerrank.com/practice-3-1
2018/2019,homework,www.hackerrank.com/si-practice-4
2018/2019,homework,www.hackerrank.com/si-practice-5
2018/2019,homework,www.hackerrank.com/sda-hw-6
2018/2019,homework,www.hackerrank.com/si-practice-7
2018/2019,homework,www.hackerrank.com/sda-hw-8
2018/2019,homework,www.hackerrank.com/sda-hw-9
2018/2019,homework,www.hackerrank.com/sda-hw-10
2018/2019,homework,www.hackerrank.com/sda-hw-11
2018/2019,exam,www.hackerrank.com/sda-test
2018/2019,exam,www.hackerrank.com/sda-ttest-2
```

Как изглеждат данните?

Състезанието https://www.hackerrank.com/contests/2023-2024-2/challenges показано чрез browser:



... и конкретна задача от състезанието например *cycles-components*...

Цикъл в компоненти

locked

Problem | Submissions | Leaderboard | Discussions

Намерете броя на свързаните компоненти в графа, в които има цикъл.

Графът ще бъде **ненасочен**. За да се установи цикъл в ненасочен граф, трябва да се спази следното правило, за да се избегне тривиалния цикъл - нямаме право да се връщаме във възела, от който сме дошли. Тоест, ако във възел 1 сме стигнали от възел 0, то нямаме право да се върнем във възел 0.

Никога няма да има повече от 1 ребро между 2 възела.

Спрямо тези правила, свързан граф с 2 върха никога няма да има цикъл.

Input Format

На първия ред ще получите едно число T - броя на заявките

За всяка заявка ще получите следното:

На първия ред ще получите две числа разделени с интервал - V и E - броя на възлите и ребрата в графа.

На следващите E на брой реда ще получите по 2 числа, разделени с интервал - $from$ и to, между кои два възела има ребро.

Constraints

$1 \leq T \leq 10$

$1 \leq V \leq 10^6$

Submissions: 111
Max Score: 100
Difficulty: Medium

Rate This Challenge:

More

Admin Options

Edit Challenge

View Submissions

```
{
  "can_be_viewed": true,
  "can_edit": null,
  "bookmarked": false,
  "dynamic": true,
  "has_started": true,
  "has_ended": false,
  "countdown_time": 0,
  "requirements_description": null,
  "max_score": 100,
  "active": true,
  "epoch_starttime": null,
  "epoch_endtime": null,
  "time_left": null,
  "factor": 100,
  "expert_solution_status": false,
  "custom_tabs": null,
  "total_count": 111,
  "solved_count": 61,
  "success_ratio": 0.5495495495495496,
  "is_editorial_available": false,
  "is_solution_unlocked": false,
  "contest_slug": "2023-2024-2",
  "topics": [],
  "user_score": 0.0,
  "track": null,
  "id": 389099,
  "slug": "cycles-components",
  "name": "Цикъл в компоненти",
  "status": null,
  "created_at": "2024-01-13T20:26:41.000Z",
  "updated_at": "2024-02-01T10:15:40.000Z",
  "kind": "code",
  "preview": "wtf",
  "category": "ai",
  "deleted": false,
  "company_id": null,
  "is_custom": false,
  "player_count": 2,
  "custom_checker_language": null,
  "checker_program": null,
  "judgebot_language": null,
  "judgebot": null,
```


Как изглеждат данните?

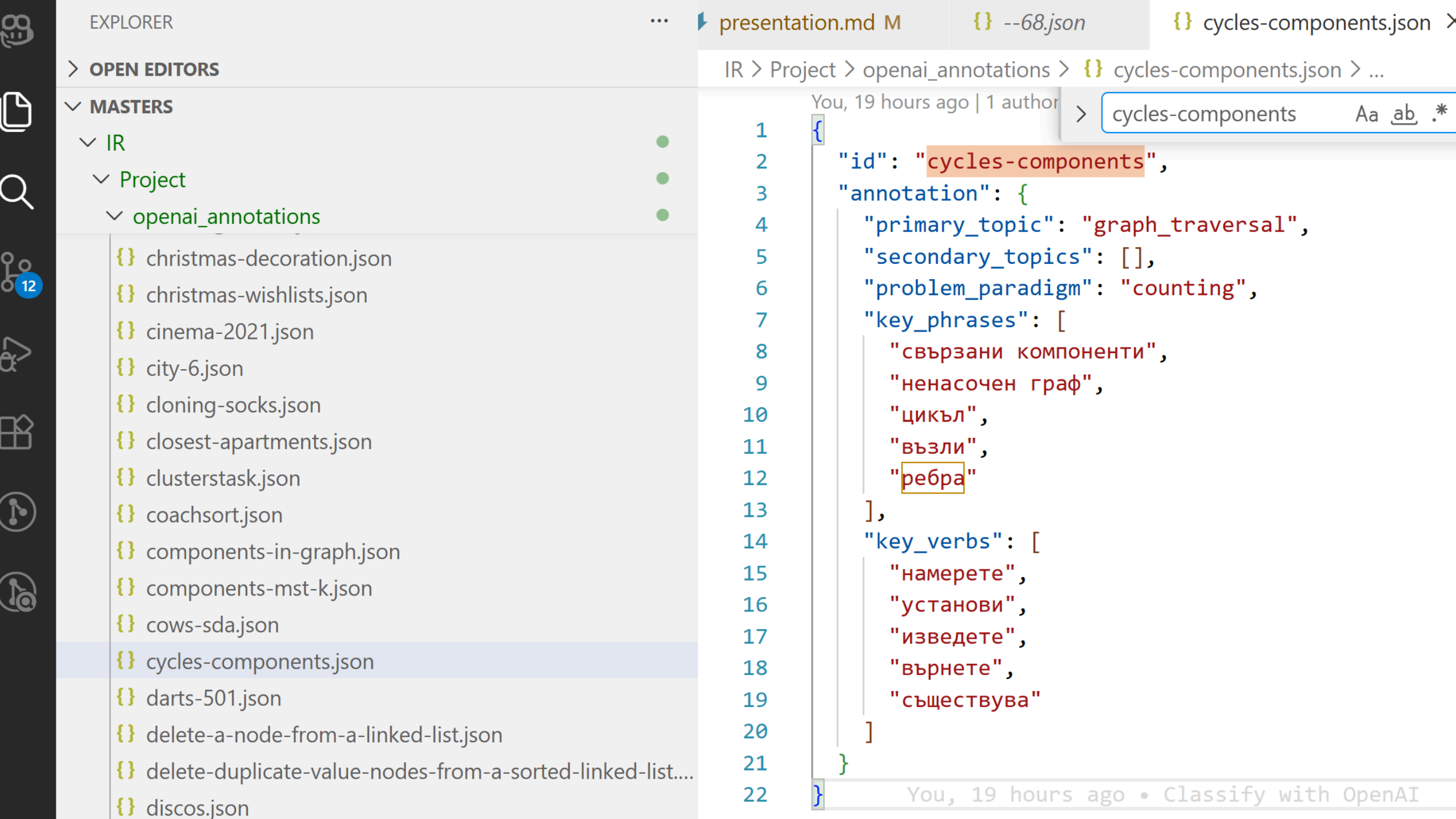
Във JSON format след scrap-ване:

Атрибутите, които ще използваме:

```
225     "input_format": "На първия ред ще получите едно число $T$ - броя на заявките
226     "output_format": "За всяка заявка $T_i$ изведете едно число на нов ред - бро
227     "problem_statement": "Намерете броя на свързаните компоненти в графа, в кои
228     "constraints": "$1 \\leq T \\leq 10$\n\n$1 \\leq V \\leq 10^6$\n\n$1 \\leq
229 }
```

Анотиране на данните

- Слабо анотиране чрез OpenAI агент
- Дефиниране на клаове задачи според категориите лекции по конспект


EXPLORER
OPEN EDITORS
MASTERS
IR
Project
openai_annotations
christmas-decoration.json
christmas-wishlists.json
cinema-2021.json
city-6.json
cloning-socks.json
closest-apartments.json
clusterstask.json
coachsort.json
components-in-graph.json
components-mst-k.json
cows-sda.json
cycles-components.json
darts-501.json
delete-a-node-from-a-linked-list.json
delete-duplicate-value-nodes-from-a-sorted-linked-list....
discos.json
presentation.md M
--68.json
cycles-components.json
IR > Project > openai_annotations > cycles-components.json > ...
You, 19 hours ago | 1 author
cycles-components
{
"id": "cycles-components",
"annotation": {
"primary_topic": "graph_traversal",
"secondary_topics": [],
"problem_paradigm": "counting",
"key_phrases": [
"свързани компоненти",
"ненасочен граф",
"цикъл",
"възли",
"ребра"
],
"key_verbs": [
"намерете",
"установи",
"изведете",
"върнете",
"съществува"
]
}
}
You, 19 hours ago • Classify with OpenAI

Как изглеждат данните? (обобщение)

- 329 текстови файла
- Съдържат условието на всяка задача в raw текстов формат
- Категория на задачата (label)

```
Total files: 329

=== Primary Topic Counts ===
  dynamic_programming                      68
  graph_traversal                          52
  sorting                                  44
  binary_search_tree                       38
  hash_table                               32
  linked_list                              25
  stack_queue                              23
  shortest_path                            20
  searching                                 8
  minimum_spanning_tree                     8
  priority_queue                            8
  eulerian_hamiltonian                      1
```

3. Избор на подход (Метод)

а. Какви основни обработки от КЛ са необходими?

- Нормализация на текста (премахване на HTML тагове, пунктуация, малки букви)
- Токенизация и лематизация чрез **Classla**
- POS tagging - филтриране по NOUN, PROPN, ADJ, VERB
- Премахване на стоп-думи (stopwordsiso за български)
- Извличане на признаци: униграми, биграми, глаголи

b. Какви допълнителни алгоритми са приложени?

- **PMI-базирано индуциране на правила** - за всеки клас се избират до 30 "cue" думи с най-висок PMI score (Експ. I, II)
- **Intent/Slot Extraction** - ръчно дефинирани cue думи за извличане на topic, paradigm, direction (Експ. III)
- **TF-IDF + Cosine Similarity** - векторно представяне на лематизиран текст и търсене на подобни задачи (Експ. IV)
- Стратифицирано разделяне train/test (80/20)

4. Експерименти върху данните (Експерименти)

I. Експеримент: Rule-based PMI класификатор

- Използване на PMI-базиран rule-based класификатор върху български текст
- NLP pipeline: Classla (tokenize, POS, lemma) + stopwords филтриране
- Признаци: леми (униграми, биграми), глаголи, key_phrases от анотации
- За всеки клас се индуцират Top-30 правила с положителен PMI score

I. Експеримент - Резултати

Метрика	Стойност
Accuracy	0.333 (22/66)
Перфектни класове (F1=1.0)	binary_search_tree, linked_list
Неточни класове (F1=0.0)	dynamic_programming, priority_queue, stack_queue, tree

I. Експеримент - Анализ по класове

Клас	Precision	Recall	F1	Support
binary_search_tree	1.00	1.00	1.00	6
linked_list	1.00	1.00	1.00	2
shortest_path	1.00	0.40	0.57	5
graph_traversal	1.00	0.30	0.46	10
sorting	0.44	0.36	0.40	11
minimum_spanning_tree	0.17	1.00	0.29	2
dynamic_programming	0.00	0.00	0.00	13

I. Експеримент - Изводи

- Класове с характерна лексика (BST, linked_list) се разпознават перфектно
- Класове дефинирани чрез техника на решение (DP, stack_queue) нямат лексикални сигнали в условието, за което съответсва F1 = 0
- Висок precision, нисък recall при graph/shortest_path - правилата са точни, но недостатъчни
- Подходът не улавя абстрактни алгоритмични концепции

II. Експеримент: PMI Cue Extraction със слети класове

- Същият PMI подход, но с **обединени класове** чрез LABEL_MAP (12 → 7 класа)
  - graph_traversal + shortest_path + MST → **graph**
  - binary_search_tree + tree → **tree_structure**
  - linked_list + stack_queue + priority_queue → **linear_structure**
- Оценка чрез **Top-K accuracy** вместо само Top-1
- 31 тестови примера

II. Експеримент - Резултати

Метрика	Стойност
Top-1 Accuracy	0.226 (7/31)
Top-3 Accuracy	0.645 (20/31)
Top-5 Accuracy	0.903 (28/31)
Cue Overlap	0.968 (30/31)

II. Експеримент - Анализ

- Top-1 accuracy е нисък (22.6%), но правилният клас попада в **Top-3 в 64.5%** и в **Top-5 в 90.3%** от случаите
- Cue overlap 96.8% - почти винаги правилният клас получава *някакъв* score
- Основен проблем: **searching** доминира като Top-1 предсказание за почти всички класове - правилата за searching са твърде общи
- Сливането на класовете намалява броя тестови примери (66 → 31), но не подобрява Top-1

III. Експеримент: Intent/Slot Extraction

```
"tree": {
    "дърво", "възел", "корен", "лист", "поддърво",
    "родител", "наследник", "предшественик", "предък",
    "дълбочина", "височина", "ниво", "бинарен",
}
```

```
"graph_traversal": {
    "граф", "обход", "обхождам", "посещавам",
    "достигам", "преминавам", "обиколка",
    "връх", "ребро", "съсед", "съседен",
    "компонента", "свързаност", "цикъл",
    "двуделен", "подграф", "инцидентен", "смежен"
}
```

III. Експеримент: Intent/Slot Extraction

- Ръчно дефинирани **slot правила** с лематизирани cue думи за български
- Извличане на структурирана информация за всяка задача:
  - **topic** (12 класа), **paradigm** (6 типа)
- Оценка на topic slot срещу OpenAI анотации (329 примера, без train/test split)

III. Експеримент - Резултати

Метрика	Стойност
Accuracy	0.307
Macro Precision	0.236
Macro Recall	0.196
Macro F1	0.186

III. Експеримент - Изводи

- Accuracy (30.7%) е сходна с PMI класификатора, но macro F1 (0.186) е по-нисък
- Ръчните cue думи са ограничени - не покриват разнообразието на текстовете
- Предимство: Можем да извлечем допълнителни атрибути

IV. Експеримент: TF-IDF Similarity Search

- Лематизиран текст - **TF-IDF** вектори (униграми + биграми)
- **Cosine similarity** между всяка задача и останалите
- Leave-one-out оценка: за всяка задача търсим Top-5 най-подобни
- Метрики: P@1, P@3, P@5, MRR, MAP@5

IV. Експеримент - Резултати

Метрика	Стойност
P@1	0.511
P@3	0.441
P@5	0.423
MRR	0.614
MAP@5	0.587
Общо заявки	329

IV. Експеримент - Анализ по класове (P@1)

Клас	P@1	Брой
binary_search_tree	0.842	38
linked_list	0.840	25
graph_traversal	0.673	52
shortest_path	0.600	20
sorting	0.500	44
dynamic_programming	0.368	68
stack_queue	0.348	23
hash_table	0.219	32

IV. Експеримент - Изводи

- P@1 = 51.1% - значително подобрение спрямо rule-based подходите (~30%)
- MRR = 0.614 - правилният клас обикновено е в първите 2 резултата
- BST и linked_list отново водят (>84%) - силен лексикален сигнал
- DP (36.8%) и hash_table (21.9%) остават трудни - условията им не споделят уникална лексика
- TF-IDF улавя по-добре контекста от ръчни cue думи, но все още е ограничен от bag-of-words представянето

5. Обобщение на резултатите

Експеримент	Метрика	Стойност
I. PMI класификатор	Accuracy	0.333
II. PMI със слети класове	Top-1	0.226
II. PMI със слети класове	Top-5	0.903
III. Intent/Slot Extraction	Accuracy	0.307
IV. TF-IDF Similarity	P@1	0.511
IV. TF-IDF Similarity	MRR	0.614

6. Бъдеща работа

- **Transformer embeddings** - използване на многоезичен модел (напр. multilingual BERT) за по-богато семантично представяне
- **Подобряване на анотациите** - ръчна проверка на OpenAI етикетите за по-надеждна оценка
- **Уеб интерфейс** - интеграция на similarity search в студентски портал за търсене на подобни задачи

Време за въпроси